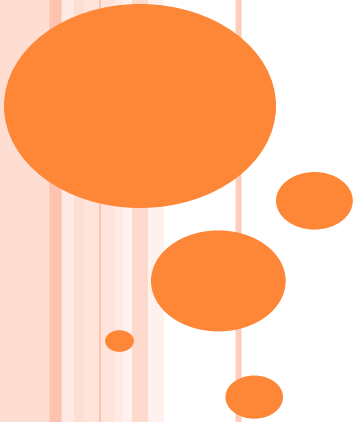
Patrones de Comportamiento

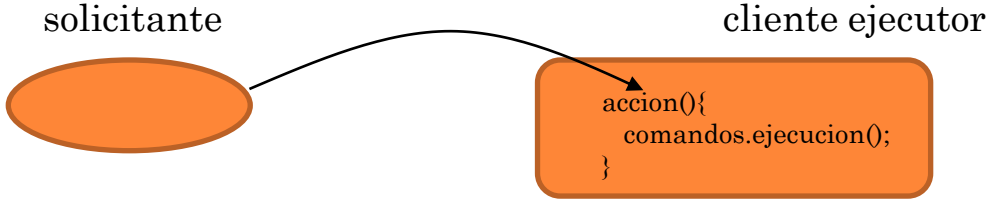


Patrón Command

- Se trata de un patrón que encapsula una acción o comandos. Es utilizado en contextos donde se requiere parametrizar objetos, a los que hay que suministrarles una operación a realizar mediante comandos.

- Los comandos son enviados desde el solicitante de la acción al que realiza la acción, manteniendo a ambos desacoplados.

- Un comando puede ser almacenado o encolado y poder ser ejecutado varias veces.



Patrón Command. Ejemplo práctico

- **Lo primero es definir una interfaz con las operaciones del comando:**

```
public interface Command {
    void execute();
}
```

- **A partir de ella, podemos tener comandos con operaciones concretas, como grabar un pedido, actualizar un stock,..:**

```
@Component
public class CreateOrderCommand implements Command {
    private final OrderRepository orderRepository;
    @Autowired
    public CreateOrderCommand(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }
    @Override
    public void execute() {
        System.out.println("Pedido creado.");
        // lógica real:
        orderRepository.save(order)
    }
}
```



Patrón Command. Ejemplo práctico

```
@Component
public class StockCommand implements Command {
    @Override
    public void execute() {
        System.out.println("Actualziando stock.");
        :
         restApi.invoke(….);
    }
}
@Component
public class SendConfirmationEmailCommand implements Command {
    @Override
    public void execute() {
        System.out.println("Email de confirmación enviado.");
        :
        EmailSender.send(…);
    }
}
```



Patrón Command. Ejemplo práctico

- **Tendríamos un componente invocador, que es el que recibe el comando o comandos, los almacena y proporciona un método para que todos sean invocados:**

```
@Component
public class CommandDispatcher {
    private final List<Command> commands;
    public void addCommand(Command command) {
        commands.add(command);
    }
    public void dispatch() {
        commands.forEach(Command::execute);
    }
}
```



Patrón Command. Ejemplo práctico. Uso

- Como cliente ejecutor, en este ejemplo utilizamos un service, que agrupa los comandos y los lanza a través del invocador.
- Al agrupar las acciones en comandos, el service se desacopla de los objetos a utilizar

```
public class PedidoService {
    private final CommandDispatcher dispatcher = new CommandDispatcher();
    private final OrderRepository repository= new OrderRepositoryImpl();


    public void crearPedido(String pedidoId) {
        System.out.println("Pedido creado: " + pedidoId);

        // Encapsulamos las operaciones como comandos
        dispatcher.addCommand(new StockCommand());
        dispatcher.addCommand(new CreateOrderCommand(repository));
        dispatcher.addCommand(new SendConfirmationEmailCommand());

        // Ejecutamos los comandos
        dispatcher.dispatch();
    }
}
```

Patrón Estrategia

- Se utiliza en contextos donde queremos aplicar diferentes tipos de algoritmos para realizar una tarea (búsqueda, cálculo,...), pudiendo elegir en tiempo de ejecución cual aplicar en cada momento.

- Evita codificar bloques if .. elseif.. para elegir el algoritmo a aplicar, lo que violaría el principio SOLID abierto/cerrado...

- Se basa en definir una interfaz con la operación a realizar y tantas implementaciones como estrategias de implementación diferentes



Patrón Estrategia. Ejemplo práctico

- **Primeramente se define una interfaz común con la operación a implementar. Por ejemplo, para aplicar una determinada estrategia de pago:**

```
public interface EstrategiaPago {
    void pagar(double importe);
}
```



Patrón Estrategia. Ejemplo práctico

- **Definiremos una clase que implemente la interfaz por cada estrategia de pago: tarjeta, paypal, etc.:**

```
public class PagoTarjeta implements EstrategiaPago {
    @Override
    public void pagar(double importe) {
        System.out.println("Validando tarjeta...");
        System.out.println("Contactando con pasarela bancaria...");
        System.out.println("Autorizando pago de " + importe);
    }
}
public class PagoPayPal implements EstrategiaPago {
    @Override
    public void pagar(double importe) {
        System.out.println("Autenticando usuario en PayPal...");
        System.out.println("Invocando API de PayPal...");
        System.out.println("Procesando pago de " + importe);
    }
}
```

Patrón Estrategia. Ejemplo práctico

- **La capa service gestionaría la estrategia a utilizar:**

```
public class PagoService {

    private EstrategiaPago estrategia;

    public PagoService(EstrategiaPago estrategia) {
        this.estrategia = estrategia;
    }

    public void realizarPago(double importe) {
        estrategia.pagar(importe);
    }

    public void setEstrategia(EstrategiaPago estrategia) {
        this.estrategia = estrategia;
    }
}
```



Patrón Estrategia. Ejemplo práctico. Uso

- **Desde una clase principal configuramos la estrategia a utilizar y se la proporcionamos al service:**

```
public class Main {

    public static void main(String[] args) {

        PagoService servicio =
            new PagoService(new PagoTarjeta());

        servicio.realizarPago(100);

        servicio.setEstrategia(new PagoPayPal());

        servicio.realizarPago(200);

        servicio.setEstrategia(new PagoTransferencia());

        servicio.realizarPago(300);
    }
}
```

Patrón Template Method

- Se utiliza en contextos donde varias clases tienen que realizar un determinado proceso formado por varios pasos comunes.
- Algunos de estos pasos son idénticos para todas las clases, pero otros se implementan de forma específica en cada una
- Se basa en definir una una clase abstracta que defina el proceso y sus pasos, proporcionando para algunos implementaciones que las clases hijas puedan heredar.
- Su principal objetivo es la reutilización de código



Patrón Template. Ejemplo práctico

- **Aplicación que procesa diferentes tipos de documentos, y se deben seguir los mismos pasos para todos:**

```
public abstract class ProcesadorDocumento {
    // Método plantilla
    public final void procesar() {
        abrirDocumento();
        leerContenido();
        procesarDatos();
        cerrarDocumento();
    }
    //métodos que sobrescribirán cada clase
    protected abstract void leerContenido();
    protected abstract void procesarDatos()
    //métodos comunes, siempre así
    private void abrirDocumento() {
        System.out.println("Abriendo documento...");
    }
    private void cerrarDocumento() {
        System.out.println("Cerrando documento...");
    }
}
```

Patrón Template. Ejemplo práctico

➢Procesador de documentos PDF:

```
public class ProcesadorPDF extends ProcesadorDocumento {
    @Override
    protected void leerContenido() {
        System.out.println("Leyendo contenido del PDF...");
    }
    @Override
    protected void procesarDatos() {
        System.out.println("Extrayendo información del PDF...");
    }
}
```

➢Procesador de documentos excel:

```
public class ProcesadorExcel extends ProcesadorDocumento {
    @Override
    protected void leerContenido() {
        System.out.println("Leyendo celdas del Excel...");
    }
    @Override
    protected void procesarDatos() {
        System.out.println("Calculando resultados del Excel...");
    }
}
```

Patrón Template. Ejemplo práctico

- **Clse demo:**

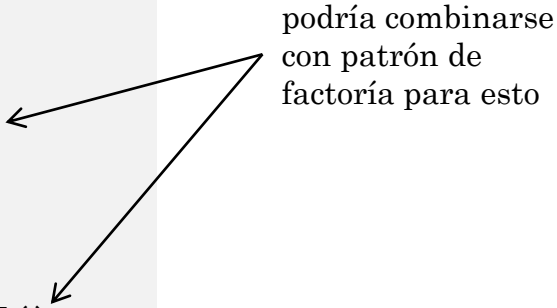
```
public class Main {

    public static void main(String[] args) {

        ProcesadorDocumento pdf = new ProcesadorPDF();
        pdf.procesar();

        System.out.println("----------------");

        ProcesadorDocumento excel = new ProcesadorExcel();
        excel.procesar();
    }
}
```



Template vs strategy

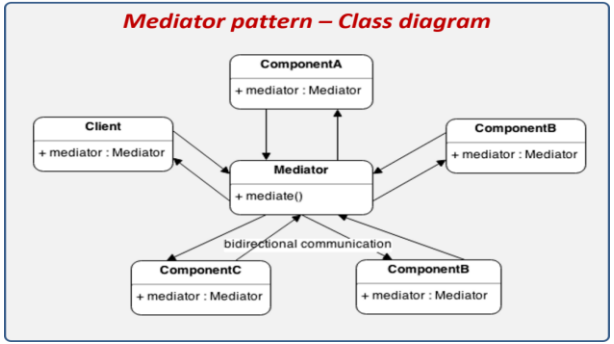
- Es fácil confundir estos patrones, pero hay una diferencia importante: mientras que Template Method establece un algoritmo fijo con pasos personalizables, Strategy permite personalizar el propio algoritmo

- A nivel de implementación, Template Method se basa íntegramente en la herencia, mientras que Strategy utiliza composición



Patrón Mediador

- Gestiona la comunicación entre un conjunto de clases para evitar el acoplamiento entre las mismas.



- Un escenario clásico de este patrón es un chat grupal donde usuarios se envían mensajes unos a otros. En lugar de establecer una comunicación directa entre objetos, se utiliza un servidor central (mediador) al cual se envían los mensajes y que luego se encarga de redirigirlos a su destinatario

Patrón Mediador. Ejemplo práctico

- **Implementaremos un chat grupal donde los mensajes son enviados a todos los miembros del grupo**

- **Para la aplicación de este patrón, definimos una interfaz con los métodos que el mediador debe implementar. En el ejemplo del chat:**

```
public interface ChatMediator {
    void enviarMensaje(String mensaje, Usuario emisor);
    void registrarUsuario(Usuario usuario);
}
```



Patrón Mediador. Ejemplo práctico

- **Un servidor de chat que implementase esa interfaz podría ser :**

```
public class ChatConcreto implements ChatMediator {
    private List<Usuario> usuarios = new ArrayList<>();
    public void registrarUsuario(Usuario usuario) {
        usuarios.add(usuario);
    }
    public void enviarMensaje(String mensaje, Usuario emisor) {
        for (Usuario usuario : usuarios) {
            if (usuario != emisor) {
                usuario.recibir(mensaje);
            }
        }
    }
}
```

Patrón Mediador. Ejemplo práctico

- **Definiríamos una clase abstracta Usuario con la funcionalidad que debería exponer un usuario:**

```
public abstract class Usuario {
    protected ChatMediator mediador;
    protected String nombre;

    public Usuario(ChatMediator mediador, String nombre) {
        this.mediador = mediador;
        this.nombre = nombre;
    }

    public abstract void enviar(String mensaje);
    public abstract void recibir(String mensaje);
}
```

Patrón Mediador. Ejemplo práctico

- **Y que un usuario concreto heredaría e implementaría de la siguiente forma :**

```
public class UsuarioConcreto extends Usuario {
    public UsuarioConcreto(ChatMediator mediador, String nombre) {
        super(mediador, nombre);
    }

    public void enviar(String mensaje) {
        System.out.println(this.nombre + " envía: " + mensaje);
        mediador.enviarMensaje(mensaje, this);
    }

    public void recibir(String mensaje) {
        System.out.println(this.nombre + " recibe: " + mensaje);
    }
}
```



Patrón Mediador. Ejemplo práctico. Uso

- **Desde otra clase se orquestaría la creación de usuarios, mediador y envío de mensajes :**

```
public class Main {
    public static void main(String[] args) {
        ChatMediator chat = new ChatConcreto();
        Usuario u1 = new UsuarioConcreto(chat, "Ana");
        Usuario u2 = new UsuarioConcreto(chat, "Luis");
        Usuario u3 = new UsuarioConcreto(chat, "Carlos");
        chat.registrarUsuario(u1);
        chat.registrarUsuario(u2);
        chat.registrarUsuario(u3);

        u1.enviar("Hola a todos");
    }
}
```

Patrón Memento

- Permite guardar y restaurar el estado de un objeto sin exponer los detalles internos del mismo a otras clases.

- Útil en escenarios en los que necesitamos tener la oportunidad de deshacer cambios.

- Los actores principales en este patrón son:

  - Originador. Es el objeto cuyo estado se quiere guardar
  - Memento. El objeto encargado de almacenar el estado de los originadores. El originador es el único que puede acceder a él
  - Cuidador. Se encarga de guardar los mementos con los estados de los objetos

Patrón Memento. Ejemplo práctico

- **Vamos a presentar un ejemplo de patrón memento para gestionar el estado de perfiles de usuario. Cada perfil se define por un usuario y una preferencia.**

- **Lo primero será crear la clase Memento, que simplemente replica los datos que se quieren guardar:**

```
public class UserProfileSnapshot {
    private final String nombre;
    private final String preferencia;
    public UserProfileSnapshot(String nombre, String preferencia) {
        this.nombre = nombre;
        this.preferencia = preferencia;
    }
    // Getters
    public String getNombre() { return nombre; }
    public String getPreferencia() { return preferencia; }
}
```



Patrón Memento. Ejemplo práctico

- **El originador tendría la siguiente implementación:**

```
public class UserProfile {
    private String nombre;
    private String preferencia;
    public UserProfile (String nombre, String preferencia) {
        this.nombre = nombre;
        this. preferencia = preferencia;
    }
    public void actualizar(String nombre, String preferencia) {
        this.nombre = nombre;
        this. preferencia = preferencia;
    }
    public UserProfileSnapshot guardarVersion() {
        return new UserProfileSnapshot(nombre, preferencia);
    }
    public void restaurar(UserProfileSnapshot snapshot) {
        this.nombre = snapshot.getNombre();
        this.preferencia= snapshot.getPreferencia();
    }
    @Override
    public String toString() {
        return "UserProfile{" +
                "nombre='" + nombre + '\'' +
                ", preferencia='" + preferencia + '\'' +
                '}';
    }
}
```

Patrón Memento. Ejemplo práctico

- **Un cuidador sería una clase encargada de almacenar objetos Memento, proporcionando métodos para guardar y obtener Mementos para restaurar :**

```
public class CuidadorManager {
    private final Stack<UserProfileSnapshot> historial = new Stack<>();
    public void guardar(UserProfileSnapshot snapshot) {
        historial.push(snapshot);
    }
    public UserProfileSnapshot deshacer() {
        if (!historial.isEmpty()) {
            return historial.pop();
        }
        return null;
    }
}
```

Patrón Memento. Ejemplo práctico. Uso

- **En el siguiente ejemplo de prueba creamos un perfil, lo modificamos y luego deshacemos los cambios para volver a su versión original:**

```
public class Main {
    public static void main(String[] args) {
        UserProfile perfil = new UserProfile("Prueba", "música");
        CuidadorManager versiones = new CuidadorManager();
        System.out.println("Perfil inicial: " + perfil);
        //guarda un Memento con el perfil inicial
        versiones.guardar(perfil.guardarVersion());
        perfil.actualizar("prueba nueva", "deportes");
        System.out.println("Actualizado: " + perfil);
        // Deshacer último cambio
        perfil.restaurar(versiones.deshacer());
        //se verán los datos del estado inicial
        System.out.println("Deshacer : " + perfil);
    }
}
```

Patrón Observer

- Se utiliza para notificar cambios de estado en un objeto a otros objetos que actúan como observadores. Desacoplando ambos grupos de objetos.

- En este patrón hay dos actores principales:

  - Subject. Es el objeto observado y el que notifica los cambios a los observadores. Mantiene una lista de los objetos observadores suscritos

  - Observer. Se trata del objeto que observa cambios en otros objetos. Incluirá un método que será ejecutado automáticamente cada vez que se reciba una notificación de cambio de estado en el subject

Se utiliza en aplicaciones de tipo reactivo cuando ciertos objetos deben "reaccionar" a un suceso ocurrido en otro

Patrón Observer. Ejemplo práctico

- La siguiente interfaz define el método que un observer debe implementar para responder a un evento generado desde el subject, el cual enviará un mensaje de texto durante la notificación:

```
public interface Observer {
    void recibir(String mensaje);
}
```

- El subject a su vez deberá implementar la siguiente interfaz para permitir el registro de observadores y lanzar notificaciones:

```
public interface Subject {
    void registrar(Observer o);
    void eliminar(Observer o);
    void notificar(String mensaje);
}
```



Patrón Observer. Ejemplo práctico

- **Una implementación concreta de subject podría ser :**

```
public class AvisosSubject implements Subject {
    private final List<Observer> suscriptores = new ArrayList<>();
    @Override
    public void registrar(Observer o) {
        suscriptores.add(o);
    }
    @Override
    public void eliminar(Observer o) {
        suscriptores.remove(o);
    }
    @Override
    public void notificar(String mensaje) {
        for (Observer o : suscriptores) {
            o.recibir(mensaje);
        }
    }
    public void nuevoAviso(String aviso) {
        System.out.println("Nuevo aviso: " + aviso);
        notificar(aviso);
    }
}
```



Patrón Observer. Ejemplo práctico

- **Una implementación de ejemplo del Observer se encargaría simplemente de mostrar el mensaje recibido:**

```
public class Usuario implements Observer {
    private final String nombre;

    public Usuario(String nombre) {
        this.nombre = nombre;
    }
    @Override
    public void recibir(String aviso) {
        System.out.println( nombre + " recibió: " + aviso);
    }
}
```

Patrón Observer. Ejemplo práctico. Uso

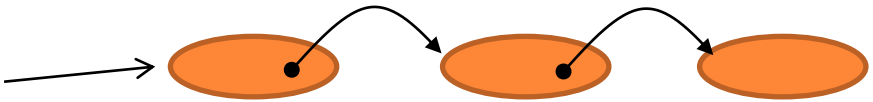
- **Otra clase crearía dos observadores y los registraría en el subject, después provocaría el evento en el subject y comprobaríamos que los observadores reaccionan:**

```
public class Main {
    public static void main(String[] args) {
        AvisosSubject avisos = new AvisosSubject ();

        Usuario pru1 = new Usuario("Prueba 1");
        Usuario pru2 = new Usuario("Prueba2");
        avisos.registrar(pru1);
        avisos.registrar(pru2);
        avisos.nuevoAviso ("Patrón Observer en acción");
        avisos.nuevoAviso("Avanzando en el curso");
        avisos.eliminar(pru1);
       //solo reacciona el único usuario que queda registrado
        avisos.nuevoAviso ("Primer usuario  ya no verá este aviso");
    }
}
```

Patrón Cadena de Responsabilidad

- Su objetivo es desacoplar el emisor de una petición de los posibles receptores.
- El objeto emisor envía la petición a una cadena de objetos receptores, la cual va pasando a lo largo de la cadena hasta que uno de ellos la atiende.
- Los actores principales en este patrón son:
  - Handler. Define una interfaz para manejar solicitudes y mantiene una referencia al siguiente manejador
  - Concrete handler. Implementa la lógica para procesar la solicitud o delegar en el siguiente
  - Client. Envía la petición al primer manejador de la cadena



Ejemplo práctico

- Vamos a presentar un sistema de soporte con tres niveles de atención.
- El ticket va pasando por cada nivel
- Este sería el handler abstracto

```
abstract class Soporte {
    protected Soporte siguiente;

    public void setSiguiente(Soporte siguiente) {
        this.siguiente = siguiente;
    }

    public abstract void manejarTicket(String tipoProblema);
}
```



Ejemplo práctico

- **Manejadores concretos:**

```
class Nivel1 extends Soporte {
    @Override
    public void manejarTicket(String tipoProblema) {
        if(tipoProblema.equalsIgnoreCase("basico")) {
            System.out.println("Nivel 1: Resolviendo problema básico.");
        } else if(siguiente != null) {
            siguiente.manejarTicket(tipoProblema);
        } else {
            System.out.println("Nadie puede resolver este problema.");
        }
    }
}
class Nivel2 extends Soporte {
    @Override
    public void manejarTicket(String tipoProblema) {
        if(tipoProblema.equalsIgnoreCase("intermedio")) {
            System.out.println("Nivel 2: Resolviendo problema intermedio.");
        } else if(siguiente != null) {
            siguiente.manejarTicket(tipoProblema);
        } else {
            System.out.println("Nadie puede resolver este problema.");
        }

    }
}
```

Ejemplo práctico

- **Manejadores concretos:**

```
class Nivel3 extends Soporte {
    @Override
    public void manejarTicket(String tipoProblema) {
        if(tipoProblema.equalsIgnoreCase("avanzado")) {
            System.out.println("Nivel 3: Resolviendo problema avanzado.");
        } else if(siguiente != null) {
            siguiente.manejarTicket(tipoProblema);
        } else {
            System.out.println("Nadie puede resolver este problema.");
        }
    }
}
```



Ejemplo práctico. Uso

- **Clase cliente:**

```
public class Main {
    public static void main(String[] args) {
        // Crear manejadores
        Soporte nivel1 = new Nivel1();
        Soporte nivel2 = new Nivel2();
        Soporte nivel3 = new Nivel3();

        // Configurar cadena
        nivel1.setSiguiente(nivel2);
        nivel2.setSiguiente(nivel3);

        // Enviar solicitudes
        nivel1.manejarTicket("basico");       // Nivel 1 lo resuelve
        nivel1.manejarTicket("intermedio");   // Nivel 2 lo resuelve
        nivel1.manejarTicket("avanzado");     // Nivel 3 lo resuelve
        nivel1.manejarTicket("imposible");    // Nadie lo resuelve
    }
}
```

Ejemplo práctico. Lambdas

```
class Handler {
    private Consumer<String> action;
    private Handler next;

    public Handler(Consumer<String> action) {
        this.action = action;
    }

    public Handler setNext(Handler next) {
        this.next = next;
        return next;
    }

    public void handle(String request) {
        action.accept(request);
        if (next != null) {
            next.handle(request);
        }
    }
}
```



Ejemplo práctico. Lambdas. Uso

```
public class Main {
    public static void main(String[] args) {
        // Definimos la cadena con lambdas
        Handler nivel1 = new Handler(req -> {
            if (req.equalsIgnoreCase("basico")) {
                System.out.println("Nivel 1 resolvió el problema.");
            }
        });
        Handler nivel2 = new Handler(req -> {
            if (req.equalsIgnoreCase("intermedio")) {
                System.out.println("Nivel 2 resolvió el problema.");
            }
        });
        Handler nivel3 = new Handler(req -> {
            if (req.equalsIgnoreCase("avanzado")) {
                System.out.println("Nivel 3 resolvió el problema.");
            } else if (!req.equalsIgnoreCase("basico") && !req.equalsIgnoreCase("intermedio")) {
                System.out.println("Nadie puede resolver este problema.");
            }
        });

        // Encadenamos
        nivel1.setNext(nivel2).setNext(nivel3);

        // Probamos
        nivel1.handle("basico");
        nivel1.handle("intermedio");
        nivel1.handle("avanzado");
        nivel1.handle("imposible");
    }
}
```



Patrón Estado

- Este patrón tiene que como objetivo que un objeto pueda cambiar dinámicamente su comportamiento al cambiar su estado, evitando uso de múltiples if/switch.
- Cada estado se representa como una clase independiente que encapsula el comportamiento correspondiente
- Los actores principales son:
  - State (Interfaz/abstracta): Define las operaciones que deben implementar los estados.
  - ConcreteState: Implementan el comportamiento específico de cada estado.
  - Context: Mantiene una referencia al estado actual y delega en él la lógica

Ejemplo práctico

- **Pedido. Tres estados posibles: Nuevo(puede pagar), Pagado(puede enviar), Enviado (no puede hacer nada)**

- **Interfaz estado:**

```
interface Estado {
    void pagar(Pedido pedido);
    void enviar(Pedido pedido);
}
```

- **Pedido (contexto):**

```
class Pedido {
    private Estado estado;
    public Pedido() {
        this.estado = new Nuevo(); // arranca como nuevo
    }
    public void setEstado(Estado estado) {
        this.estado = estado;
    }
    public void pagar() {
        estado.pagar(this);
    }
    public void enviar() {
        estado.enviar(this);
    }
}
```

Ejemplo práctico

➢Estados concretos:

```
class Nuevo implements Estado {
    public void pagar(Pedido pedido) {
        System.out.println("  Pedido pagado");
        pedido.setEstado(new Pagado());
    }
    public void enviar(Pedido pedido) {
        System.out.println("  No puedes enviar, aún no está pagado");
    }
}

class Pagado implements Estado {
    public void pagar(Pedido pedido) {
        System.out.println("  Ya está pagado");
    }
    public void enviar(Pedido pedido) {
        System.out.println("  Pedido enviado");
        pedido.setEstado(new Enviado());
    }
}
class Enviado implements Estado {
    public void pagar(Pedido pedido) {
        System.out.println("  No puedes pagar, ya está enviado");
    }
    public void enviar(Pedido pedido) {
        System.out.println("  Ya está enviado");
    }
}
```

Ejemplo práctico. Uso

```
public class Main {
    public static void main(String[] args) {
        Pedido pedido = new Pedido();

        pedido.enviar(); // ❌ No puedes enviar
        pedido.pagar();  // ✅ Pedido pagado
        pedido.enviar(); // ✅ Pedido enviado
        pedido.pagar();  // ❌ No puedes pagar, ya está enviado
    }
}
```



Patrón Visitor

- Este patrón permite definir nuevas operaciones sobre una estructura de objetos sin modificar sus clases.
- El objeto debe estar preparado para aceptar visitantes
- Los actores principales son:
  - Visitor (Interfaz). Declara una operación visit para cada tipo de objeto de la jerarquía.
  - ConcreteVisitor. Implementa esas operaciones.
  - Element (Interfaz). Declara accept(Visitor v).
  - ConcreteElement. Implementan accept llamando al visitante adecuado.
  - Client. Tiene la estructura de objetos y aplica el visitante

Patrón visitor. Ejemplo práctico

- **Figuras geométricas**

```
sealed interface Figura permits Circulo,Rectangulo{
    void accept(FiguraVisitor visitor);
}
final class Circulo implements Figura {
    double radio;
    Circulo(double radio) { this.radio = radio; }

    @Override
    public void accept(FiguraVisitor visitor) {
        visitor.visit(this);
    }
}
final class Rectangulo implements Figura {
    double ancho, alto;
    Rectangulo(double ancho, double alto) {
        this.ancho = ancho; this.alto = alto;
    }

    @Override
    public void accept(FiguraVisitor visitor) {
        visitor.visit(this);
    }
}
```



Patrón visitor. Ejemplo práctico

➢Visitante y visitantes concretos

```
interface FiguraVisitor {
    void visit(Circulo c);
    void visit(Rectangulo r);
}
class AreaVisitor implements FiguraVisitor {
    @Override
    public void visit(Circulo c) {
        double area = Math.PI * Math.pow(c.radio, 2);
        System.out.println("Área del círculo: " + area);
    }

    @Override
    public void visit(Rectangulo r) {
        double area = r.ancho * r.alto;
        System.out.println("Área del rectángulo: " + area);
    }
}
class DibujarVisitor implements FiguraVisitor {
    @Override
    public void visit(Circulo c) {
        System.out.println("Dibujando un círculo de radio " + c.radio);
    }
    @Override
    public void visit(Rectangulo r) {
        System.out.println("Dibujando un rectángulo " + r.ancho + "x" + r.alto);
    }
}
```

Patrón visitor. Ejemplo práctico. Uso

➢Main

```
public class Main {
    public static void main(String[] args) {
        List<Figura> figuras = List.of(
            new Circulo(5),
            new Rectangulo(4, 6)
        );

        FiguraVisitor areaVisitor = new AreaVisitor();
        FiguraVisitor dibujarVisitor = new DibujarVisitor();

        System.out.println("== Cálculo de áreas ==");
        figuras.forEach(f -> f.accept(areaVisitor));

        System.out.println("\n== Dibujado ==");
        figuras.forEach(f -> f.accept(dibujarVisitor));
    }
}
```

